

Q1 and Q2 (from 2025-02-01 until 2025-07-31).

<!-- TIP: To update the table below, create the schedule in a temporary spreadsheet, and then copy/paste the rows into an online markdown generator (<https://www.google.com/search?q=copy-table-in-excel-and-paste-as-a-markdown-table>) -->

Designer	Design Pair	
-----	-----	
Bonnie Tsang	Annabel Gray	
Julia Miocene	Nick Brandt	
Lina Fowler	Emily Bauman	
Dan Mizzi-Harris	Torian Parker	
Nick Leonard	Jesse Hoek	
Amelia Bauerly	Austin Regnery	
Camellia Yang	Jeremy Elder	
Becka Lippert	Pedro Moreira da Silva	
Alex Fracazo	Sunjung Park	
Chad Lavimoniere	Michael Le	
Taylor Vanderhelm	Mark Unthank	
Graham Bachelder	Timothy Noah	
Jesse Young	Ilonah Pelaez, Veethika Mishra	
Alyssa Trinh	Sascha Eggenberger	
Libor Vanc	Michael Fangman	

SECTION: product/ux/how-we-work/figma-provisioning.md

Figma provisioners/deprovisioners are found in the Tech Stack YAML file.

While multiple departments have team members with Editor roles as paid seats, the UX department is the business owner.

Figma seats

Seat types

At GitLab, we use two of three available Figma seat types: Viewer-restricted and Full.

Viewer-restricted: Free seat. People with a Viewer-restricted seat can view and comment on files to collaborate. All GitLab team members are added with this seat type during onboarding.

Full: Paid seat. People with a Full seat can create and edit files. A paid seat requires provisioning.

Note: We do not use the Viewer role as this can result in unexpected seat upgrades. All viewer roles should be marked as Viewer-restricted, which is also the default seat type when a user is added.

Learn more about roles in Figma.

Seat options

Figma has three different products that seat types can apply to:

Design: A Full seat allows a team member to edit Design files. When you upgrade someone to a Full Design seat, they'll automatically get upgraded to a Full Dev Mode seat at no additional cost.

Dev Mode: Dev Mode is included with a Full Design seat and is also available as a standalone paid seat.

FigJam: A Full seat allows a team member to edit FigJam files.

Provisioning

Access request issues

Every approved paid Figma seat must have a corresponding access request issue with budget approval. Seat upgrades can only happen with Okta access through Google groups and not by using the Figma admin or Figma's sharing tools.

Provisioners need to be pinged in the access request issue to grant access. As a provisioner, ensure:

- The team member's manager has approved access.

- The team member understands what seat option they need.

- A Viewer-restricted role is often enough when a team member only needs to view and comment on files to collaborate. All team members start with a Viewer-restricted role, so no action may be needed.

- The difference between a Design and FigJam seat can be confusing at first. Ensure the team member knows which seat they need in order to perform their role.

- A team member may have the same, or a different seat type for Design and FigJam.

- There is budget approval. Billing group admins should work with their Finance partners to ensure seat counts are not going over the maximum spend allocated for the fiscal year.

Once an access request issue has manager approval:

1. Sign in to Google Groups.

1. Click My groups.

1. Click the name of the group that you want to add the user to. All Google groups which manage users in Figma application are formatting as okta-figma-xxxxx-users and include descriptions with access details. A user only needs to be added to the group that matches the highest level of permissions the user needs.

1. Next press the People tab on the left side and select Members.

1. To add a member press the Add Members button. When a member is added from the group it may take a few minutes for the sync to happen between Google and Figma. Once the sync happens the user's details will be updated in the Figma admin. If you are unable to access a specific group, please reach out to another provisioner for help.

1. Request help in the figmamaintainers Slack channel to have a Figma application admin (different from provisioner) confirm the user's status in Figma's admin and to assign a billing group and workspace (optional). Currently, it's not possible to automate assigning a team member to a billing group or workspace in Figma.

Quarterly audits

Our billing cycle includes quarterly true-ups. Each quarter, admins receive an email informing them of the upcoming invoice. This gives them an opportunity to review added seats before approving the new invoice.

To audit seats:

1. Review all new seats and ensure each team member (or guest) has an approved access request issue.
1. Downgrade anyone who shouldn't have a paid seat (see steps under deprovisioning).
1. Mark your billing group as Reviewed.

Once confirmed, the Figma business owner will:

1. Lock in the final invoice amount.
1. Reach out to our Figma rep to share total seat counts for R&D, Marketing, and Sales. These will be added to line items on the invoice.
1. Ensure the invoice is uploaded to Coupa and goes through our full Procurement process.

Deprovisioning

A user will automatically be fully deprovisioned when they are no longer with GitLab. Additionally, a paid seat can manually be downgraded to an unpaid Viewer-restricted seat.

When a member is fully removed:

Anyone with 'can edit' access to the member's files will be able to continue editing and can move the files.

Their draft files stay within the organization and Figma admins can view and manage the files.

For more information, refer to Figma's [Remove people from an organization](#) documentation.

To downgrade a user to an unpaid seat:

1. Sign in to Google Groups.
1. Click My groups.
1. Click the name of the group that you want to remove the user from. (Note that the group will correspond with their current access type.)
1. Next press the People tab on the left side and select Members. This will show you all the members currently in the group.
1. Hover over a user and select the checkbox that appears. Then click the remove member button (Looks like a circle with a horizontal line across) at the top of the member list. When a member is removed from the group it may take a few minutes for the sync to happen between Google and Figma. Once the sync happens the user's details will be updated in the Figma admin. If you are unable to access a specific group, please reach out to another provisioner for help.
1. Request help in the figmamaintainers Slack channel to have a Figma application admin (different from provisioner) confirm the user's status.

SECTION: product/ux/how-we-work/product-design-manager-pairs.md

Product Design Manager Pairs

Product Design Manager pairs give PDMs a regular touchpoint with their peers and additional exposure to other product areas outside their own.

- New pairs are connected every six months.
- A PDM who is a manager of managers should not be paired with one of their direct reports.
- If we have an odd number of people, we'll have one 3-person partnership.
- Pairs decide what to share and when; there is no predefined schedule or guidelines.

Collaboration suggestions

- Share team processes and ways of working.
- Walk your pair through your product areas to familiarize them with it.
- Talk through ongoing and upcoming work happening in your area. This can uncover opportunities for cross-stage collaboration.
- Discuss strengths and skillsets on your teams to connect product designers looking to grow in certain areas with product designers who are already strong in those areas.

Schedule

This is the rotation schedule for FY26-Q2 and Q3 (2025-05-01 until 2025-10-31).

<!-- TIP: To update the table below, create the schedule in a temporary spreadsheet, and then copy/paste the rows into an online markdown generator (<https://www.google.com/search?q=copy-table-in-excel-and-paste-as-a-markdown-table>) -->

Product Design Manager Product Design Manager	
----- -----	
Jacki Bauer	Gina Doyle
Emily Sybrant	Caitlin Steele
Justin Mandell	Chris Micek
Paul Wright	Rayana Verissimo

SECTION: product/ux/how-we-work/cross-stage-design-collaboration/_index.md

How do we determine if the topic is cross-stage design

There are several points during the design process where we might identify cross-stage collaboration needs. It is recommended to involve potential partners as early as possible. It is best to determine if there is a collaboration need quickly rather than have to backtrack later when it is too late.

During the discovery and research phase, it is essential to pay attention to the user journey or other types of research reports; from the research insights, we can assess if the user is

using multiple features that cross-stages or if the pain points involve features owned by different teams.

During the design phase, there could be signs that we need to contact other teams for potential collaboration. First, the target user is the primary persona of another stage; second, the use case you are targeting is part of a JTBD belonging to another stage; third, the feature you are designing exists in a page/area belonging to another stage; and last, some part of the user's journey is using some features that belong to another stage.

For Pajamas contributions, it is a collaboration-first approach. When designing a component, we want to ensure that the component is suitable for all areas. When we design reusable features and components, such as search or filtering, it is better to collaborate with more than one designer to consider the frequency and complexity of these features in the product.

How can we find the right DRI for a particular feature

GitLab has a wide range of product features. Several resources can help us identify the Directly Responsible Individual (DRI) for each feature:

- Which feature belongs to which group: use the category page and the feature page
 - Who is responsible for the feature: use the team page
- Ask in the ux-coworking channel if you can't find the answer using the above resources

When should we contact the other team(s)

During the planning phase, if you can anticipate a certain amount of effort will be needed from other team members, it's better to contact them as soon as possible. Suggestions:

- Contact the other team(s) before problem validation if we can anticipate that they may want or be able to contribute to the research.
- Contact the other team(s) after problem validation in relation to the research insights if a collaboration opportunity was uncovered during the research.

If, for some reason, another relevant team was not involved before problem validation, make sure to involve them as early as possible in the design discovery and exploration process and provide opportunities for the other team(s) to provide feedback on the ideal future-state design, to avoid duplicative work.

What kind of information would be helpful for teams for better cross-stage collaboration

At the problem validation stage, the goal of the collaboration is to make sure the problems are the right ones to solve by all parties, so it's recommended to have the following information:

- Jobs To Be Done (JTBD) and personas
 - Related previous research
 - A research plan (or at least some research questions or a hypothesis to be answered)
- At the design stage, the goal is to design a solution together that matches all user needs and workflows. Therefore, it is essential to have clear documentation of the following:
- How the feature works: both from the user's point of view and a technical point of view.
 - The precise problem statement, JTBD, and persona(s)
 - Relevant user feedback/ UXR insights and data

- Design scope and urgency: this helps all teams figure out a prioritization and collaboration plan

What needs to be agreed upon for better cross-stage design collaboration

To execute effective collaboration, come to an agreement on the following with your team and relevant design partners:

- Prioritization: It is ideal if all teams agree on when this problem needs to be addressed, and both the leading team and supporting team should plan together. However, if one team needs the feature sooner to solve confirmed user pain points, the team can be the DRI and move forward with an MVC. At the same time, keep the discussions ongoing for further design, research, and development plans for the next step.
- Effort and timing: Based on the prioritization, all teams should agree on clear effort needs from each team and the time frame for the project.
- Who leads the collaboration: There is no definite answer to who is leading the design and implementation. In general, consider the urgency of the need and the capacity of both the leading designer and the designer who provides support. For example, if two different teams will lead the implementation and design, the implementation should be at most six months after the design effort is done.

Cross-stage design collaboration frameworks

The collaboration methods can be flexible. You can collaborate like design pairs. Or use formal frameworks to help designers work together, consider using those, especially when there are more than two designers. Design Pod is a framework that has clearly defined roles and responsibilities and is tasked with achieving a high-impact business goal. If the design collaboration is more aimed at exploitation without a clear business goal, consider using a Design Jam instead.

Tips for cross-stage collaboration

- Keep a single source of truth in one issue/epic.
- Have a list of todos and be sure to check them off when complete so that it is clear and straightforward to everyone what the current status is and what still needs to be done.
- Have a "parking lot" issue to put future (post-MVC) ideas/ discussions in.
- Have a sync session or record a video walking through the prototype to show the context of the product.

Always assume positive intent (and other communication tips per our handbook). And keep in mind that all GitLab users are essential, not just those that our stages tend to prioritize.

SECTION: product/ux/how-we-work/cross-stage-design-collaboration/design-pod.md

What is a Design Pod

A Design Pod is a design team comprised of two or more Product Designers and other relevant counterparts from various Stage Groups. It has defined roles and responsibilities and is tasked with achieving a high-impact business goal that will specifically affect the user experience of

the product across Stage Group workflows. Members will come together to tackle a singular design problem following GitLab's output design principles and Product Designer Workflow processes. Once this goal is achieved the Design Pod will disband allowing the Stage Group DRI to drive the project to completion.

Roles and Responsibilities (all roles are required)

DRI

The pod's DRI should be the Product Designer that is most closely related to the problem being addressed's Stage Group or Section. The DRI will be Accountable for guiding the pod's design direction. While they'll work collaboratively with the rest of the Design Pod, as the DRI, they'll be in charge of decision-making and driving the best strategy for completing the pod's work. It's highly recommended that the DRI at least be a Senior Product Designer as the role will require a high level of leadership and organizational skills.

Product Manager (Sponsor)

The Product Manager that is most closely related to the addressed problem. The Product Manager's Stage Group or Section will need to approve the pod and should also prepare to participate in the oversight. The Product Manager should also expect to be consulted as they often have unique knowledge or insights into the problem space.

Product Design Manager or Staff Product Designer of the DRI

The Product Design Manager or Staff Product Designer of the DRI is Responsible for working closely with the Design Pod to ensure they are progressing and have what they need to succeed. In addition, they may be responsible for helping free up their time to participate in the pod and assisting in recruiting other pod members or securing any tools that may not be available.

Members

Other Design Pod members will consist of Product Designers or Product Managers from other Stage Groups that have some relationship to the addressed problem. Depending on their level of participation, they'll be responsible for attending meetings synchronously or asynchronously regularly, sharing information learned from the Design Pod with their peers, gathering feedback from their peers, bringing that feedback, and completing necessary tasks to help the Design Pod progress and succeed. Members must work closely with their manager and Stage Group's Product Manager to ensure they will have at least 30% of their typical time per Milestone freed up to participate in the pod. Use the RACI model to help assign pod member roles based on interest/capacity.

Guidelines

- The DRI and Product Manager, Design Pod sponsors, are responsible for preventing the proliferation of Design Pods and ensuring there is a real need for the Design Pod to assemble.
- A member and DRI should not be a part of more than one concurrent Design Pod in any role.
- It is highly recommended that anyone in the Design Pod work with their Manager to ensure that they will have at least 30% of their time per Milestone freed up to participate in the pod and that the pod work will not adversely affect their Stage Group's needs.

- Providing accurate updates on the design status is crucial; Design Pod updates might involve a larger group of stakeholders. Be sure the following points are clearly addressed:
 - Communicate which stage of the design process the pod is in, for example: exploration, refinement, validation etc.
 - What type of feedback is needed, for example: product directions, feature suggestions, user interaction, visual design, etc?
- During the exploration design process, consider using sketches or wireframes to indicate that work is still in progress as a concept or incomplete.
- While regular sync meetings are likely necessary, consider adopting async ways to keep each pod member informed; for example: use weekly Geekbot updates in the pod Slack channel.

Process

- Preparation
 - The DRI is responsible for creating a Design Pod Issue or Epic as the SSOT
 - Name it after the problem that is being addressed but keep it brief.
- Apply ~UX, the primary DRI's ~group::<name here>, and ~DesignPod labels
 It should be public unless there is a specific reason to keep it private.
 The DRI is responsible for creating a Slack channel (with dpod prefix) that is public to the company.
- Assemble the pod team and share the formation of the pod in any appropriate Slack channel(s) to encourage participation.
 - All members together determine how the pod will work collaboratively, whether through consistent synchronous calls or asynchronous periodic check-ins. And add it to the Issue/Epic description.
 - If synchronous calls were determined to be the way, schedule the call and invite all pod members. Set any consulted or informed members to be optional participants, and be sure to share the meeting video with everyone through your Slack channel and any other relevant channel (e.g. ux, product).
 - Define the addressed problem (both user problems, product, and engineering challenges)
 - Define the goal of the Design Pod
 - Define the Design Pod team and how they will participate using the RACI model.
 - Gather any existing research or data necessary for driving the pod forward to successful completion.
 - Gather existing design proposals
 - Define What success looks like (if necessary, gather metrics that will tell you success or not, for example, a positive result from qualitative research or a quantitative research goal)
 - Organize activities that should provide incremental progress.
 - Workshops (asynchronous or synchronous)
 - Competitive Analysis/Walkthroughs, etc.
 - Figma design collaboration
 - Research (e.g. problem or solution validation)
 - Communicate the results
 - Communicate status regularly in the Design Pod's slack channel
 - Consider regular updates to the progress to whats-happening-at-gitlab, ux, and related Stage Group(s) slack channels (take the multi-modal communication approach as a guideline)
 - Disband the Design Pod
 - Celebrate. Being able to close a Design Pod is definitely a thing to celebrate!
 - Update the Issue/Epic with the final result.
 - Close the Issue/Epic

- Archive the slack channel.
- Delete the recurring calendar meeting.

Participating in an existing Design Pod

Suppose you are interested in participating in an active Design Pod. In that case, it is recommended that you first communicate with your manager and then discuss your involvement with the Design Pod's DRI. Remember that they may not be in a place to accept new members. However, if the DRI has approved your involvement, you've worked with your manager and Stage Group's Product Manager to ensure you can participate at least 30% of your time per Milestone. You can add yourself to the Design Pod members list by creating an MR against the specific Design Pod handbook page.

Past Design Pods

- (RCA: Role Based Access Control (RBAC) Design Pod, Authentication & Authorization)[<https://gitlab.com/gitlab-com/gitlab-ux/ux-rca/-/issues/3>]

SECTION: product/ux/jobs-to-be-done/_index.md

The material in this page and related pages (Playbook, Beyond the Playbook) draws from Jim Kalbach and his book, "The Jobs to be Done Playbook".

For practical JTBD research guidance, see the playbook. For a deep dive into each aspect of a job canvas, review the Anatomy of a JTBD canvas.

Note: The previous JTBD source (yml file, internal only) is being replaced. Teams should track JTBD work in FigJam until a new handbook page is created.

Main Jobs to micro jobs

When talking about Jobs to be Done, we're often talking about different levels of jobs. It's important to note the differences in terminology between these levels so that you and your stakeholders can communicate effectively.

Main Jobs

A Main Job is a means to an end. It's an act that will be performed and should have a clear end state (the "done" part of JTBD). That is why we write jobs in the pattern Verb + Object + Clarifier when writing job statements.

Example: Buy a new home

Small jobs

Small Jobs are more practical and correspond to a process or workflow. They answer the

question, "How does the job get done?" in the context of the Main Job and moves the user closer to accomplishing their goal.

Example: Put in an offer on a house

Micro-jobs

Micro-jobs are the small tasks a user may undergo to accomplish their small job and Main Job. Micro-jobs should be self-explanatory and easy to understand without much context.

Example: Decide how much you're going to offer in relation to the asking price.

It's important to be able to identify and correctly place jobs at the right altitude as you work through the Jobs to be Done process. It will help keep you focused on the Main Job and allow you to quickly incorporate (or discard) new information that you hear during interviews into your job steps.

SECTION: product/ux/jobs-to-be-done/jtbd-beyond-the-playbook.md

Using Jobs to be Done in mixed methods research

JTBD can be used in many UX research methods, from surveys to usability benchmarking. Once you validate your JTBD through interviews and create your JTBD Canvas, you can use pieces of the canvas to inform numerous GitLab processes. For example, you can use the needs and circumstances of your jobs to help you increase the realism and readability of your solution validation tests. Likewise, the main jobs can help you create recruitment screener options to find the right participants for your study.

Using JTBD to help build Recruitment Screeners

An effective recruitment screener collects information about the respondent without revealing too much information about the study they may participate in. Typically, a screener would have a question asking the participant's job title and perhaps a question about common tasks they perform. JTBD can help by using the job performer and their main job to create valid screening options. When picturing an ideal candidate, try to identify the matching job performer. Use that performer's main job instead of a job title, and we can use fewer screening questions to reveal the same information as before.

For example: Say we have a study that needs security professionals using the vulnerability report page.

Using GitLab's list of user personas, we could create a list of job titles used by security personas to generate a list of more than 10 titles, such as Security Analyst, Security Operation Engineer, Security Consultant, Application Security Engineer, etc. This list would be very cumbersome for a respondent to answer and may not reflect the accurate day-to-day role of your ideal candidate.

Using the JTBD for the Secure and Software Supply Chain Security stages, we could identify

two or three ideal jobs/job performers for our study. Look at these job statements for example: "I identify risk in my org's assets" and "I address detected business-critical vulnerabilities." Additionally, we can include jobs from other workflows in the Secure section if we want to include invalid options to weed out unfit candidates. This screener question would be a handful of options that participants can select, and the result is a participant that accurately reflects your ideal candidate.

To make it easier, you can save all of the main jobs for your stage as a template and allow only selected ones for each study you have, like in this example. If participants do not match their main job, that can signal you to go back and accurately check how you discovered your JTBD.

Using JTBD to help write Solution Validation Scripts

If you conduct solution validation to test a design, you will likely make a script containing tasks for your users to attempt and provide feedback on. A significant factor in the quality of your feedback is how realistic the tasks are for the users without biasing them with the obvious solution. One way to do that is by utilizing the circumstances, job process, and needs of the job you are building for.

You can follow these steps to incorporate JTBD within your script:

1. Identify the main job your user is trying to accomplish when using your feature. The goal of every service or product should be to provide a solution for the user's problem. So, identify which job your design is helping to solve.

1. It may also be helpful to identify potentially invalid candidates and use their job statements to create screener questions to disqualify those respondents.

- Identify the circumstances, process, and needs for that job. This information should be included in the JTBD Canvas when discovering your jobs.

1. Use the circumstances to establish the scenario(s) at the beginning of your tasks. This provides the participants' context for the tasks, similar to a user's context when performing their role.

- For example, a previously discovered JTBD Canvas for the Security Policy group found that users implement security controls when "A new security policy was published in their organization." To turn this circumstance into a scenario for an unmoderated test, we can say, "In this scenario, your organization recently published a new security policy with the parameters provided below. Your task is to implement the necessary security control(s), so your assets comply with the security policy."

1. Create solution validation tasks by using the job process and smaller jobs within it. This will avoid biasing users with information they may not have otherwise. Since JTBD uses language identified during user research, it gives users a more natural experience.

- For example, the first step in implementing security controls is "gathering the security policies." To turn this job step into a task, we can say, "First, please gather the necessary security policies." Participants will read this and naturally understand their task - and we don't have to reveal where we want them to go.

1. Use your job's needs to create the post-test evaluation questions. As you craft your wrap-up questions, you can relate the task to the participant's larger goal or the desired outcome.

- For example, the outcome that a policy creator wants most is to "reduce the likelihood of a potential threat in my organization's assets." To turn this into a question, we can say, "On a scale from 1-7, where 1 is not useful at all, and 7 is extremely useful, how useful was this

feature for you in reducing the possibility of a security threat in your organization?" You may also want to use metrics from the usability benchmark to get a quantitative comparison.

Using JTBDs to Inform a Heuristic Evaluation

Heuristic evaluations can help team members understand the user's current experience with the product. These evaluations uncover insights that may require direct changes to improve usability or further research to increase organizational understanding of a problem. Heuristic evaluations usually take 1 to 2 meetings between a Product or UX stakeholder and a UX Researcher.

You can follow these general steps to integrate the JTBD Framework into your heuristic evaluation:

1. Identify the job the user is trying to accomplish with your feature. The goal of any product or service is to provide a solution for the user's (or customer's) problem - so identify which of your stage's jobs the design is helping solve.
1. Identify the process and needs for that job. This information should be included in the JTBD Canvas when discovering your jobs.
1. Create a planning document for your heuristic evaluation similar to this template.
1. Use the job process to identify the steps to take during the evaluation process. This is the general workflow you should follow.
1. Use the circumstances to establish the scenarios to create for the evaluation. Sometimes it requires following the steps again, but from a different perspective.
1. Use the related jobs and collaborate on a consensus for which jobs are out of scope.
1. Explore GitLab's list of heuristics or use the well-known Nielsen Norman 10 Usability heuristics. Determine which heuristics apply to your evaluation.
1. Create questions to ask each other during the evaluation by re-phrasing the applicable heuristics.
 - For example, one of the heuristics is "User control and freedom". We can turn this into a question by saying, "On a scale from 1-7, where 1 is not useful at all, and 7 is extremely easy, How easy was navigating to and from the tooling?"
1. Designate who is moderating the evaluation and who is acting as the user. The "user" will move through the UI to complete the tasks while the moderator evaluates the progress and asks questions. Both stakeholders are encouraged to provide feedback, but the moderator must ensure all tasks are evaluated.

-->

SECTION: product/ux/jobs-to-be-done/jtbd-canvas-anatomy.md

The Domain: Where do you want to innovate?

Start by identifying the relevant area within your Stage Group to help determine who you're innovating for and what they need to accomplish.

Job Performer: Who do you want to innovate for?

A Job Performer is the person executing a specific job, distinct from their job title. After identifying your domain, focus on one Job Performer aligned with the Main Job.

Key characteristics:

- Describes an individual role, not a job title
- Avoids compound descriptions (no AND/OR)
- Remains high-level, not a detailed persona
- Simple and directly connected to the target job

| Good Examples | Bad Examples |

|-----|-----|

| New home buyer | Millionaire real estate investor (describes circumstances, not the job) |

| Author | Steven King (too specific) |

| Code reviewer | Software Engineer (job title) |

[Optional] Related Job Performers: Who affects our Job Performer?

Identifying other actors who relate for your Domain can provide valuable context and aide in selecting a primary Job Performer. Once a Job Performer is selected, these related Job Performers can be documented and explored with their own canvases later if needed.

Job Performers vs. Personas

While user personas represent job titles (like "Software Developer"), Job Performers focus on specific tasks within roles. One persona may handle multiple Main Jobs (coding, reviewing, maintaining infrastructure), and different personas may share the same Job Performer role. For example, various roles might review code, but we'd use a single "Code Reviewer" Job Performer to understand that specific task.

Main Job: What is the Job Performer trying to get done?

A Main Job is a timeless, functional goal that serves as the focus for innovation, representing what the Job Performer wants to accomplish independent of any specific solution.

Key characteristics:

- Begins with an action verb
- Has a clear end state
- Avoids compound descriptions (no AND/OR)
- Remains solution and technology agnostic
- Excludes adjectives (which belong in Outcomes)
- Stays broad enough to allow innovation

Structure: [verb] + [object] + [optional clarifier]

- Verb: Action to accomplish
- Object: Target of the action
- Clarifier: Additional context or conditions

| Good Examples | Bad Examples |

|-----|-----|

| Buy a new home within 10 minutes of my work | Purchase the house at 123 Main Street for less than asking (Too specific) |

| Write a book | Be a best selling author (Aspirational, not a job) |

| Ensure code changes meet organizational standards | Review Merge Requests efficiently (Technology-specific, uses adjective) |

Related Jobs: What else is the Job Performer trying to get done?

Consider 3-6 other goals the Job Performer has within the Domain. They should be distinct objectives and formulated at a similar level of detail. These Related Jobs can be explored with their own canvases later if needed.

Aspirations: What does the Job Performer aspire to become?

Aspirations are typically 1-3 "be" goals representing the Job Performer's desired personal growth from completing the Main Job. They sit at the top of the canvas, hierarchically above the Main Job.

Example hierarchy for home ownership:

- Job Performer: New homeowner
- Aspirations: Be happy with home life, Be part of a local community
- Main Job: Acquire a new home
- Related Jobs: Finance a new home, Move homes, Sell old home

Job Steps: How does the Job Performer get the job done?

Job Steps are the sequential objectives a Job Performer must complete to accomplish their Main Job. They form a Job Map, with each step being high-level rather than individual tasks. Keep steps broad enough to apply to all performers of the job.

Key characteristics:

- Begins with an action verb in first person
- Avoids compound descriptions (no AND/OR)
- Remains solution agnostic
- Flows left to right through stages
- Sub-steps stack vertically within each stage

| Good Examples | Bad Examples |

|-----|-----|

| Decide where to look for a new home | Ask Richard what neighborhoods are popular (Too specific to one scenario) |

| Determine selection criteria | |

| Seek new homes | |

| Transfer home ownership | |

Outcomes: How does the Job Performer measure success?

Outcomes are the subjective criteria Job Performers use to gauge success. A Main Job typically has 50-100 Outcome statements.

Key characteristics:

- Begins with a directional verb (minimize, increase, etc.)
- Includes a measurable unit (time, effort, likelihood)
- Ends with job-specific qualifiers
- Avoids compound statements (no AND/OR)
- Remains solution agnostic

| Good Examples | Bad Examples |

|-----|-----|

| Minimize the time it takes to identify a potential new home | Find the best home quickly (No direction, "best" isn't specific) |

| Reduce the number of compromises made when deciding on a new home | Have the most attractive house on the street (No direction, subjective measure) |

| Minimize the distance to the place of employment | |

Emotional/Social Aspects: How does the Job Performer feel and want to be perceived?

Emotional and social aspects represent the experiential side of the job, helping shape how solutions should be delivered to meet Job Performers' personal needs.

Key characteristics of Emotional aspects:

- Begins with "feel" or "avoid feeling"
- Indicates a specific emotion
- Avoids compound statements (no AND/OR)
- Remains solution agnostic

Key characteristics of Social aspects:

- Begins with "appear as" or "avoid appearing as"
- Indicates social perceptions
- Avoids compound statements (no AND/OR)
- Remains solution agnostic

| Aspect Type | Examples |

|-----|-----|

| Emotional | Feel in control of the home acquisition process |

| | Avoid feeling uncertain about new home selection |

| Social | Appear as a good future neighbor |

| | Avoid appearing unknowledgeable about the new home acquisition process |

Understanding these aspects helps design solutions that address both functional and personal needs - like adding time-saving features for programmers concerned about appearing efficient.

Job Differentiators: What factors influence how the job gets done?

Job Differentiators are circumstances that affect how the job is performed, typically involving time, manner, or place.

Key characteristics:

- Begins with "if"
- Shows range of options using "versus/vs"
- Avoids compound statements (no AND/OR)
- Remains solution agnostic

| Good Examples |

|-----|

| If the Job Performer is single vs. married |

| If the Job Performer has young children vs. children out of the house |

| If the potential new home is local vs. far away |

These differentiators can help narrow the Main Job's scope (e.g., "get energy in the morning" vs. "get energy in the afternoon at work").

SECTION: product/ux/jobs-to-be-done/jtbd-playbook.md

This playbook enables all GitLab team members to conduct JTBD research. For a deep dive into each aspect of a job canvas, review the Anatomy of a JTBD canvas. Reach out to a UX Researcher or JTBD expert if you have any questions.

A Note on Time, Confidence, and Validation

Running through the playbook process typically requires at least one quarter. Timelines vary based on job complexity and team JTBD experience.

Creating an unvalidated, assumptive JTBD canvas can be faster and valuable for team alignment. However, even with high domain knowledge, Job Performer interviews are recommended to validate Outcome statements.

Confidence Levels:

- High/Verified: Validated through Job Performer interviews
- Medium/Somewhat Assumptive: Based on existing user research
- Low/Assumptive: Based on domain knowledge without research

References

- JTBD overview
- Anatomy of a JTBD canvas

Playbook Process

Create an issue

Create an issue in UX Research Project using the discover-jtbd issue template. Tag your UX Researcher or ask for support in the uxresearch Slack channel.

Workshop 1: Scope Definition

Using the JTBD Canvas Template, you will define your Domain, Job Performer, and Main Job through a one-hour workshop with your Product Design DRI, Product Manager, and optionally Engineering and UX Research team members.

Exercise 1: Domain Selection (10 min)

When considering the Domain, ask yourself: Where do you want to innovate?

1. Brainstorm (5 min): List potential domains. Sections, Stage Groups, or Categories may be a good starting point.
1. Select (5 min): Choose one domain and add it to the canvas.

Exercise 2: Job Performer Selection (10 min)

When considering the Job Performer, ask yourself: Who are you innovating for? Where is there a need in our market? Who are we passionate about serving? What's relevant or core to our business strategy?

1. Brainstorm (5 min): List actors/stakeholders who relate to the selected domain.
1. Vote (5 min): Each workshop participant votes
 - <3 participants: 2 votes each, you can vote on the same item
 - >3 participants: 1 vote each
1. Move winner to canvas

Exercise 3: Main Job Selection (15 min)

When considering the Main Job, ask yourself: What has strategic value? What are we passionate about? What can we best support? What aligns with our business, and what is the Job Performer's core objective?

1. Brainstorm (10 min): List all jobs your selected Job Performer tries to accomplish
1. Vote (5 min): Each workshop participant votes
 - <3 participants: 2 votes each, you can vote on the same item
 - >3 participants: 1 vote each
1. Move winner to Main Job section
1. Move others to Related Jobs section

Exercise 4: Aspirations Selection (15 min)

When considering aspirations, ask yourself: What does our Job Performer aspire to become after completing the Main Job? What are their ultimate goals beyond completing this task?

1. Brainstorm (10 min): List what Job Performer wants to become after completing Main Job
1. Vote (5 min):
 - <3 participants: 6 votes each, you can vote on the same item

- >3 participants: 3 votes each
1. Move top 3 to Aspirations section

Conduct Investigation Interviews

Conduct 5-10 one-hour interviews with representative Job Performers to understand and validate the bottom half of the JTBD Canvas: Job Map, Emotional/Social aspects, Outcomes, and Job Differentiators. Add more interviews if needed until patterns emerge.

Structure:

- Moderator and notetaker present
- Notetaker documents Job Steps during interview
- Review canvas with participant for accuracy

Interview Goals:

- Map Job Steps
- Identify desired Outcomes
- Discover Related Jobs
- Understand Job Differentiators

See the Figjam template interview guide for helpful prompts.

Complete Canvas

After interviews, synthesize the collected information into your JTBD Canvas, focusing on common patterns and discarding outliers. Consider collaborating with a UX Researcher for their expertise in data synthesis.

Exercise 1: Complete Job Map

1. If per-interview Job Maps were not created during interviews, create them now:
 - Capture steps chronologically by reviewing interview notes/videos
 - Rewrite steps following the Job Step guidelines
1. Create unified Job Map:
 - Group steps into Beginning, Middle, and End categories
 - Organize related steps into vertical stacks
 - Label stacks with a high-level, single-word verb descriptor

Exercise 2: Construct Outcome Statements

1. Identify how the Job Performer would measure the success of completing the Main Job and create a sticky for each success measure
1. Format following the Outcome guidelines
1. Aim for 50-100 statements for later validation

Exercise 3: Synthesize Emotional and Social Aspects

1. Emotional Aspects:

- Extract references to how the Job Performer feels when performing the Main Job
- Format per guidelines

2. Social Aspects:

- Extract references to how the Job Performer wants to be perceived when performing the Main Job
- Format per guidelines

Exercise 4: Gather Job Differentiators

1. Identify factors that affect how the Job Performer completes Main Job
1. Format per guidelines

Workshop 2: Map and Prioritize

Using your Job Canvas, you will select your Job Steps, Emotional & Social Aspects, and Job Differentiators through a one-hour workshop with your Product Design DRI, Product Manager, and optionally Engineering and UX Research team members.

Exercise 1: Review and Select Job Steps (15 min)

When considering the most important job steps, ask yourself: Which steps are most challenging yet crucial to success?

1. Review Job Map (5 min)
 - Verify accuracy against interview data
 - Make necessary adjustments
1. Vote on Critical Steps (10 min)
 - <3 participants: 2 votes each, you can vote on the same item
 - >3 participants: 1 vote each
1. Discuss votes to align on priorities
1. Identify top voted steps
1. Copy final Job Map to canvas, marking selected steps

Exercise 2: Select Emotional Aspect Statements (10 min)

When considering emotional aspects, ask yourself: Who are you innovating for? Where is there a need in our market? Who are we passionate about serving? What's relevant or core to our business strategy?

1. Review (5 min)
 - Share emotional aspects with team
 - Ensure understanding
1. Vote (5 min)
 - <3 participants: 6 votes each, you can vote on the same item
 - >3 participants: 3 votes each
1. Arrange votes in priority pyramid
1. Confirm team alignment on top choices
1. Move top emotional aspects to canvas

Exercise 3: Select Social Aspect Statements (10 min)

Repeat Exercise 2 to select the Social Aspect Statements.

Exercise 4: Select Job Differentiator Statements (10 min)

Repeat Exercise 2 to select Job Differentiator Statements.

.. Congratulations you have successfully created a validated JTBD Canvas! ..

Outcome Opportunity Scores Survey

Run this survey using the qualtrics template to prioritize your Outcome Statements. It is recommended to work with your UX Researcher to craft and delivery this survey.

For each Outcome Statement, ask:

1. How important is this (Outcome) to you?
2. How well is this (Outcome) currently being satisfied?

Requirements:

- Screen for participants who perform the Main Job
- Target ~50 qualified responses
- Include both GitLab and non-GitLab users
- Use 10-point scale (1='very low', 10='very high')

Resources:

- Analysis spreadsheet

Calculating Scores

The analysis spreadsheet template offers a step by step guide to exporting data from a Qualtrics survey and generating a ranked list of Outcome statements with opportunity scores.

Opportunity scores run from 0-20. Even scores below 10 indicate room for improvement - prioritize highest scores within your range.

1. Satisfaction gap: Importance - Satisfaction
1. Opportunity score: Importance + Satisfaction gap

Score	Priority Level
-----	-----
16-20	Immediate action needed
10-15	High priority
7-9	Monitor
·6	Lower priority

Generate Job Stories

Job Stories synthesize data from your Job Canvas to encapsulate customer pain points without prescribing solutions. They help align development with company vision and strategy. Each domain typically has 3-5 Job Stories.

Key characteristics:

- Evidence-based from Job Canvas data
- Specific about the pain point
- Builds empathy through context
- Aspirational but achievable
- Self-evident and well-researched

Pain points must:

- Express needs, not solutions
- Be concrete, not abstract
- Be measurable, not anecdotal

Job Story Format

1. When I [at this Job Step] + [under the conditions of these Job Differentiators]
2. I want [this New Ability/customer imperative]
3. So I can [reach these Outcomes] + [and have these Emotional/Social Aspects]

Ways to use Job Stories

- Generate How Might We (HMW) statements
- Define Design Sprint challenges
- Create testable MVP hypotheses:
 - We believe that [job performers]
 - Will achieve [desired outcome]
 - While performing [job step]
 - Using [proposed solution].
 - Success will be evidenced by [specific measure]
- Ensure issues are solving validated problems
- Define usability testing criteria

<!-- Figure out how to incorporate this into UX Theme workshop

Align on Ranked Outcomes: Workshop

Here's where the magic happens. If you have the ranked Outcomes by opportunity score from your survey, you have a list of the areas that will be most impactful to your users' experience of your product. Now is the time to meet with your stakeholders and discuss the top ranked Outcomes. Our FigJam template has a workshop template and guide you can use to help address each Outcome statement.

A few things to think about during the workshop:

- Are the Outcomes already being addressed with current and planned work? If not, should they be incorporated into future planning?
- Where in the product is each Outcome done? Are there specific pages, controls, or parts of the UI that we can focus on?
- Who owns each Outcome? Are there other teams that 'own' some of the areas of opportunity? Who can you speak with to see if they are working on this Outcome already?
- What can be done to achieve the stated Outcome? Think of some potential solutions.

The result of the workshop should be that the team understands the top opportunities for that Main Job and Job Performer, and has discussed ways to address them.

You and your team now have a list of the most important areas to focus on - a list that you can share with other teams and stakeholders (even customers!) to help bring clarity and confidence to your product roadmap.

The list of opportunities will remain relevant for a long time (until you release changes to your product that address the opportunities), because they aren't based on particular features or solutions, but larger Outcomes that users want to achieve. Re-survey your Job Performers about certain Outcome statements after your team has made some changes, in order to see if the opportunity score has decreased (meaning you've made positive progress to fulfilling the desired Outcome).

-->

<!-- TODO: Connect to UX Themes (in a future MR) -->

SECTION: product/ux/jobs-to-be-done/validated-canvases-and-opportunity-scores.md

This page acts as the single source of truth for the record of Jobs to be Done canvases that have completed the GitLab Playbook process.

For each canvas / job performer we list each outcome statment that received a score of 10 or higher. As a reminder, we perceive opportunity score as having a severity as explained at the end of the determine opportunity scores section of the playbook.

Code Author

- Validated by: Ben Leduc-Mills (Create)
- Quarter Validated: FY24Q4
- Primary Teams: Create:Remote Development, Create:Code Creation, Create:Editor Extensions
- Link to JTBD Canvas
- Link to full survey results

Top Opportunities

Outcome Statement Opportunity Score
----- -----
Maximize my confidence when checking that new code is working as expected in production.

11.76 |
Minimize the time it takes to reproduce a bug/problem.	11.70
Minimize the time it takes to identify errors and bugs in my code.	11.00
Decrease the time it takes to run test cases.	11.00
Maximize my confidence that CI/CD pipelines are running smoothly.	10.85
Decrease the time it takes to fix errors and bugs in my code.	10.81
Increase the chance that documentation is kept up to date.	10.76
Minimize the errors when integrating new code into the existing system.	10.69
Increase the chance that the pipeline passes.	10.17
Minimize the effort it takes to review my own code for security and performance.	10.04

Code Reviewer

- Validated by: Ben Leduc-Mills (Create)
- Quarter Validated: FY24Q4
- Primary Teams: Create:Code Review
- Link to JTBD Canvas
- Link to full survey results

Top Opportunities

Outcome Statement	Opportunity Score
Minimize the time it takes me to understand the context of a merge request (as a reviewer).	11.09
Increase my ability to assess the larger infrastructure impact of the proposed changes (during a code review).	10.06
Increase the efficiency with which I can examine proposed code changes.	10.02

Repository Manager

- Validated by: Ben Leduc-Mills (Create)
- Quarter Validated: FY24Q4
- Primary Teams: Create:Source Code
- Link to JTBD Canvas
- Link to full survey results

Top Opportunities

Outcome Statement	Opportunity Score
Minimize the effort when searching for a specific change in the code base.	11.28

SECTION: product/ux/learning-and-development/_index.md

Welcome to Learning and Development for the UX Department at GitLab! You are welcome to explore content here for now. We'll update this page when we add this content to GitLab Learn.

The resources in this page are meant to support product designers, researchers, technical writers, and their managers to explore, learn and grow at their own pace. We aim to collect content that spans various skill levels, as well as various levels of depth/commitment. It is recommended that the UX Department engage with the resources here to help them have a successful journey at GitLab and in their career as a whole.

Most of the resources here are free but any content requiring payment can be reimbursed following the GitLab reimbursement policies.

Growth and Development

GitLab provides team members with the growth and development benefit to access external resources to develop their career. To take advantage of this benefit, review the company handbook page and follow the process.

Managers should add approved growth and development programs to the department spreadsheet tracker in order to track departmental budget.

Mentoring and coaching

College-level course involvement

- GitLab-sponsored Tufts University Human Factors/Engineering Psychology Capstone Course

Product design

Books

- The Design of Everyday Things by Don Norman

LinkedIn Learning

- Making User Experience Happen as a Team
- Interaction Design: Flow

Design strategy

Books

- Hacking Growth: How Today's Fastest-Growing Companies Drive Breakout Success by Sean Ellis
- Better Onboarding by Krystal Higgins
- Escaping the Build Trap: How Effective Product Management Creates Real Value by Melissa Perri
- Inspired: How to Create Tech Products Customers Love by Marty Cagan
- Empowered: Ordinary People, Extraordinary Products by Marty Cagan, Chris Jones

Videos

- Design Complex Applications: A Framework

Courses

- Design MBA-Learn how to align design and business goals without sacrificing your values

Design systems

Books

- Building Design Systems: Unify User Experiences through a Shared Design Language by Sarrah Vesselov, Taurie Davis
- Expressive Design Systems by Yesenia Perez-Cruz

UX Research

Handbook Links

- Discovery Resources for Product Managers

LinkedIn Learning

- UX Research: Lean Experimentation - This is a great overview of Lean UX and discusses business impact, iteration, hypothesis writing, experiment design, lean interviewing and more.
- Surveys and Questionnaires for UX Projects - This one might be a review for some of you but it's a good overview of what to know about surveys.
- UX Deep Dive: Analyzing Data - This one isn't about quantitative research, but it is about how to properly analyze your qualitative data to get to good insights.

Other Resources

- Data-Driven Design: Quantitative Research for UX

Jobs to be Done

Handbook Links

- JTBD Resources for Product Managers

Books

- The Jobs to be Done Playbook by Jim Kalbach

Growth and Experimentation

Books

- Hacking Growth: How Today's Fastest-Growing Companies Drive Breakout Success
- Badass: Making Users Awesome
- Better Onboarding

Blog posts

- Small experiments, significant results and learnings

- Why iterative software development is critical

Handbook

- Minimum Viable Experiment
- How Growth launches experiments
- Growth Experiments Knowledge Base

Videos

- GitLab Growth experiment discussion: Adding the "Invite members" option in assignee dropdowns
- Growth Coffee & Learn: Anuj · Author Growth Hacking for Dummies
- UX Forum: New user onboarding strategy
- UX Forum: New User Onboarding
- UX Forum: Growth - Continuous Onboarding
- UX Forum: An improved Trial Signup for GitLab.com
- UX Forum: Promoting the value of locked features
- UX Forum: Authoring a Pipeline for the first time
- UX Forum: Simple changes, significant results
- How to launch product experiments at GitLab
- UX collaboration on Growth experiments at GitLab

Other Resources

- Growth Design
- Practicing Growth Design
- What is a Growth Designer?
- Growth Designers Resources
- Growth Design Case Studies
- The Psychology of Design
- 10 insights into user-centered growth design
- A/B Testing: Concept != Execution
- Design Like a Scientist

UX writing

- Microcopy: Discover How Tiny Bits of Text Make Tasty Apps and Websites by Niaw de Leon

Leadership and Communication

Books

- Radical Candor by Kim Scott
- Articulating Design Decisions by Tom Greever
- The Making of a Manager: What to Do When Everyone Looks to You by Julie Zhuo

Handbook Links

- Leadership at GitLab: Recommended articles
- Leadership at GitLab: Recommended books

Unfiltered Videos

- GitLab Design Talks: Rayana Verissimo talks about career development and design leadership
- GitLab Design Talks: Lorie Whitaker talks about the role of a Staff UX Researcher
- GitLab Design talks: Marcel van Remmerden talks about career development and design leadership
- GitLab Design talks: Pedro Moreira da Silva talks about navigating the IC career path in UX

Other Resources

- Staff Design: Navigating the individual contributor career path

LinkedIn Learning

- Leading with Intelligent Disobedience

Time management and productivity

Books

- Deep Work - Rules for Focused Success in a Distracted World by Cal Newport
- Make time by Jake Knapp and John Zeratsky

Behavioral Psychology in Design

Books

- Hooked: How to Build Habit-Forming Products by Nir Eyal
- Thinking, fast and slow by Daniel Kahneman
- Indistractable by Nir Eyal

Interaction and UI Design

Courses

- Shift Nudge

Books

- Refactoring UI by Adam Wathan and Steve Schoger
- About Face 3: The Essentials of Interaction Design by Alan Cooper, Robert Reimann, David Cronin

Other GitLab resources

Handbook Links

- Product Management Learning and Development - a comprehensive list of internal and external resources that Product Managers at GitLab can use to build their skills.

Book Clubs

- From time to time, we run an internal book club on a book from one of our resource lists. Join the conversation in ux-book-club.
- GitLab Book Clubs project
- Leadership Book clubs

SECTION: product/ux/learning-and-development/tufts-university-capstone-course.md

At GitLab, we believe everyone can contribute. In that spirit, members of the UX Department can volunteer to sponsor a project for the Tufts University Human Factors Engineering/Engineering Psychology (BS) senior-level students to take on during the spring semester.

We engage, support, and recognize students during the capstone design experience. As an industry sponsor, GitLab works with the students to solve an existing problem that is relevant to both Tufts and our organization.

The Tufts Capstone is a great opportunity for GitLab Team Members looking to share their knowledge and learn from each other. Some of the benefits include:

- Improve your mentoring and coaching skills by working with students who are new to GitLab
- Improve your communication skills, specifically around the concepts we work with every day using clear language that simplifies complexity
- Get more experience with public presence by supporting the design and research community.

Capstone course overview

The program duration is 15 weeks. Students participate in groups of 4-5 to solve human factors problems or design challenges set by industry sponsors.

Throughout the semester, students will deliver professional-level work, including weekly reports and presentations (with peer review), article reviews, and sponsor meetings/reviews. User research methods and principles will be applied along with project management and planning. The outcome of this course includes students creating a final project video, poster, and presentation which will be presented to the class, department, and sponsors.

How the project works

Each year in the fall, the project lead recruits GitLab Team Member volunteers and solicits proposals for projects.

At the start of the spring semester (usually in January), the participating students vote on which proposal they would like to work on.

The volunteer team begins meeting weekly with the students once they have decided on a proposal. Weekly meetings continue throughout the spring semester (In 2025, the students' spring semester is January 15, 2025 through May 9, 2025).

At the end of the semester, the students present their results to an internal audience at GitLab (which is open to the entire org). They also present their final project to their professor.

Following the end of the semester, the project lead solicits feedback from volunteers and students, documents the results of the project, and arranges for the following year's project leadership.

How to volunteer

If you're interested in volunteering for the Spring 2025 semester:

1. Reach out to your manager for awareness and to discuss how participation might impact your capacity.

1. Add yourself to the list of volunteers.

1. (Optional but highly encouraged!) Brainstorm projects that would be a good fit for the students.

Volunteer team composition

For best results, we are looking for 3-5 volunteers from GitLab's UX team, at least 1 of whom is a Product Designer and 1 of whom is a UX Researcher.

Project proposals

Do you have a project that has been in the backlog for a while now? It could be a good one for the students to take on! In general, the best projects include:

- A loosely defined scope so that the students can bring their creativity. The instructor may suggest the students scope it down so they can complete it in the 4 month semester. For example, improving the GitLab CI/CD onboarding flow. This topic provides enough flexibility to go through the entire design process but also allows the students to scope it down to evaluate specific areas of the onboarding flow and target certain users.
- An opportunity to perform discovery research, design (prototype), and solution validation.
- Async-first approach.

Each project proposal should include a summary with:

- A title: A concise and catchy name for the project. This is one of the first things the students will see when choosing their project for the semester.
- Description: A summary of the problem, including a problem statement and general background about the concepts that would make sense to beginner, non-GitLab users.

You can propose project ideas for the Spring 2025 semester in the comments in our volunteer recruitment issue.

Project ideas

You can use this list of previous projects considered for the Tufts University capstone project

to help you get started selecting a project or to help brainstorm ideas.

{{% details summary="Help GitLab increase their community contributions from developers and designers" %}}

Open-source principles are at GitLab's heart and are one of the reasons why GitLab users love the product. We depend on the community to contribute to our product. Yes, that's right! A developer, or even a designer, could contribute their own code or feature design, and it can end up being added to our product. We want as many community contributions as possible. This project is actually based on an issue that was brought up by a community contributor. You will focus on understanding how we can improve GitLab to motivate the community to make contributions. You'll be running research with real users to understand developers' and designers' motivations to contribute code or designs. You'll also create mockups (you can use our Pajamas UI kit in Figma) based on what you learn and then validate them with real users.

{{% /details %}}

{{% details summary="How are developers using runners and why are they important to them?" %}}

Runners are at the core of Continuous integration and deployment (CI/CD), one of GitLab's primary offerings. Thousands of users use our GitLab SaaS Runners, managed by our own GitLab team members. For those companies who run their own instances of GitLab and cannot use gitlab.com, bringing and managing their own runner is required in order to use CI/CD features. We've done extensive research around the management and observability needs of platform engineers who are responsible for those runners, but what isn't clear is how developers (and others with lower permissions) make use of runners and what runner information is crucial for them to complete their jobs. You will focus on understanding the problem at hand by running research with real GitLab developers. You'll also create mockups Pajamas UI kit in Figma based on what you learn and then validate them with users.

Interview developers to understand how they are interacting with runners today and what type of information they need to know about runners (primary issue: Problem validation: How do developers (and others with the same, or less, permissions) interact with runners?). Create mockup proposals to present this data that will replace the Project>CI/CD>Runners view and validate them with real developers.

{{% /details %}}

{{% details summary="How are users interacting with project-level analytics and are they satisfied?" %}}

Various personas make use of our project-level analyti